Universidad Nacional Autónoma de México

Licenciatura en Informática

Desarrollo de Aplicaciones Móviles

Cristian Cardoso Arellano

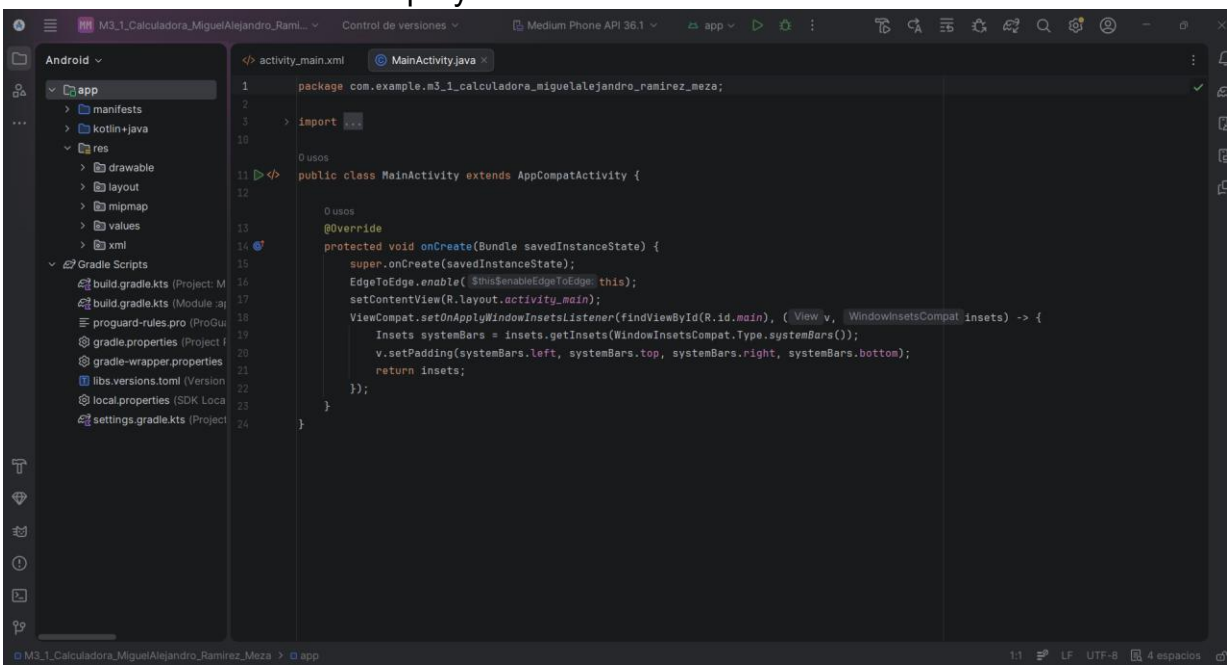
Miguel Alejandro Ramírez Meza

Unidad 3 Actividad 1

Fecha: 08 de marzo de 2026

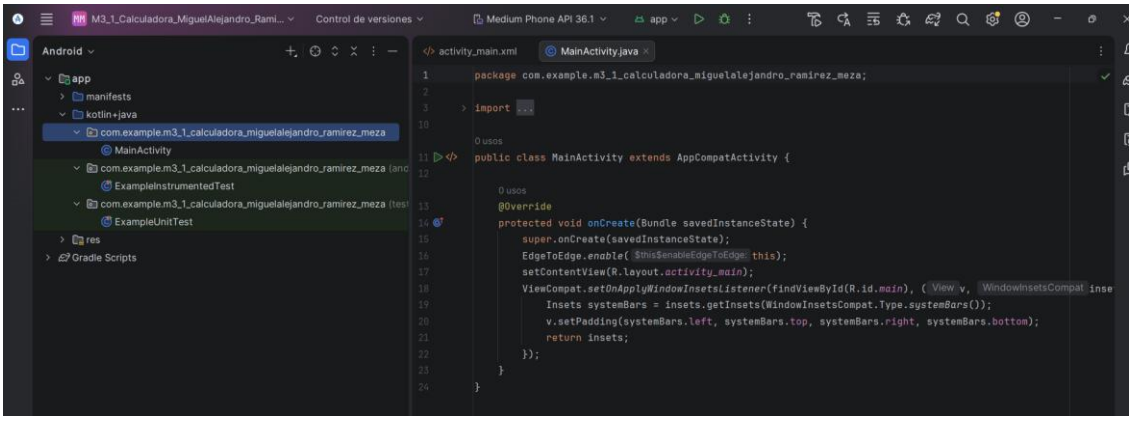
1. **Realiza un programa de tipo calculadora sin interfaz gráfica, la calculadora deberá de incluir las 4 operaciones básicas (suma, resta, multiplicación, división)**
2. **Creación de interfaz calculadora. Nota: interfaces de programación. https://docs.oracle.com/javase/tutorial/java/landl/createinterface.html**
3. **El resultado de las operaciones deberá de desplegarse en la salida estándar del logcat.**
4. **Deberá utilizar los conceptos revisados en la Unidad 3.**
5. **La calculadora deberá de validar NaN**
6. **Puede utilizar las clases bases de: https://github.com/CristianCardosoA/ANDROID-2696/blob/main/calc.zip**
7. **Subir un sólo archivo en formato PDF en el repositorio de github creado por el alumno.**
    - **android_2026_m3_01.pdf**
    - **código fuente.**

Primero iniciamos un nuevo proyecto en Android Studio

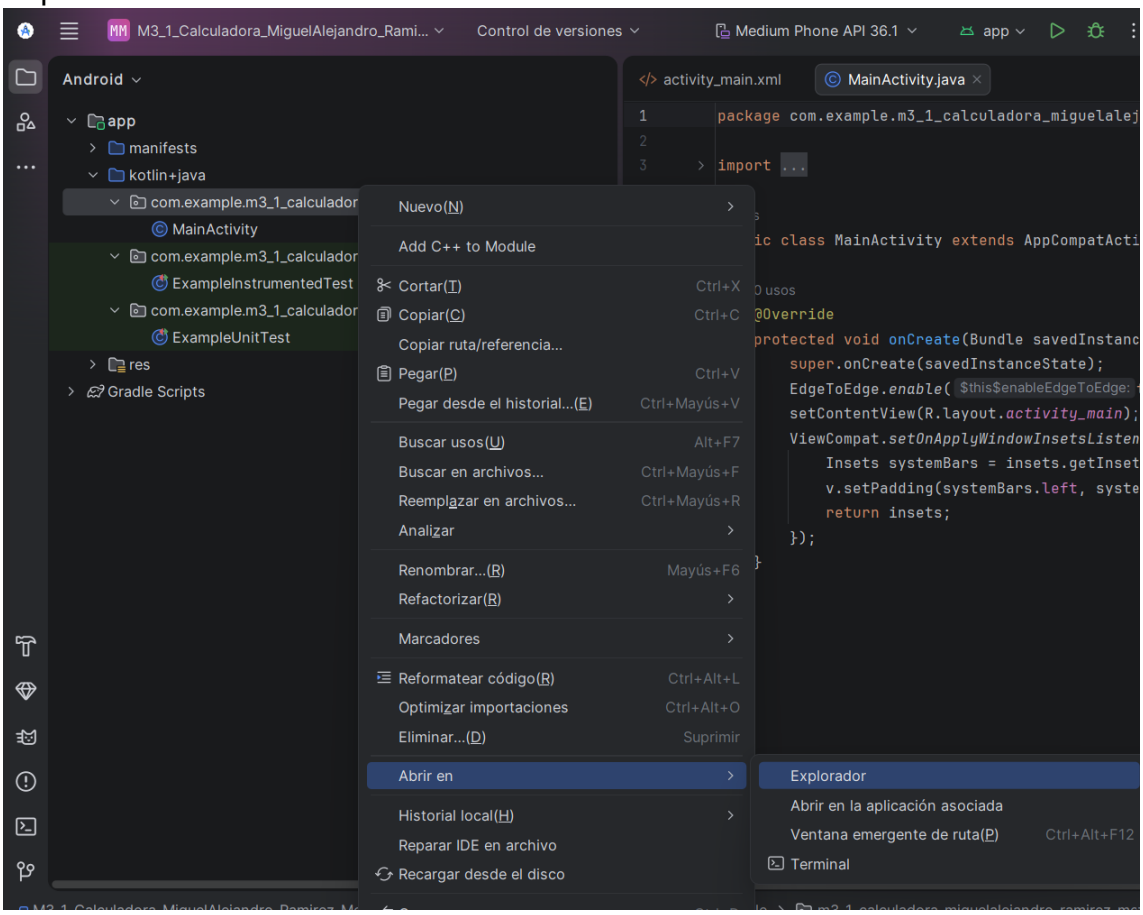


Para integrar los archivos descargados de la calculadora en el proyecto debemos desplegar la carpeta kotlin + java

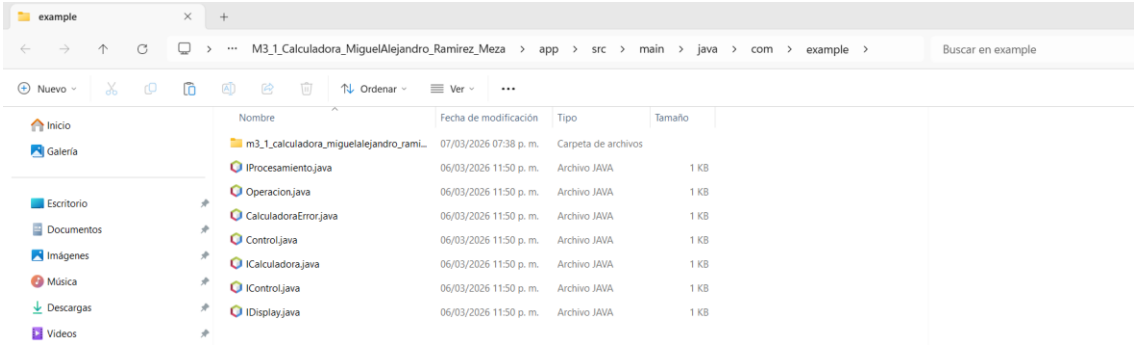
Posteriormente, seleccionamos la carpeta con el nombre del proyecto que no tiene etiquetas adicionales y que se compone por el código MainActivity



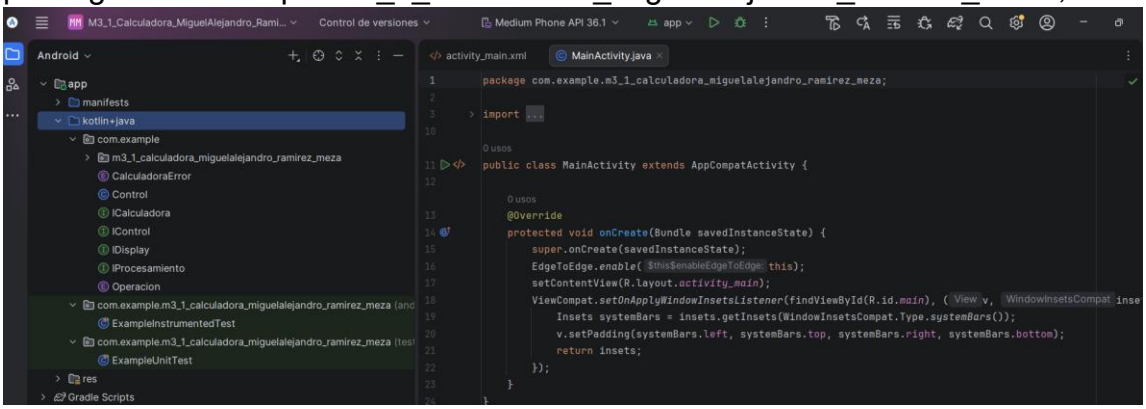
Aquí, debemos de dar clic derecho sobre la carpeta y seleccionar Abrir en > Explorador



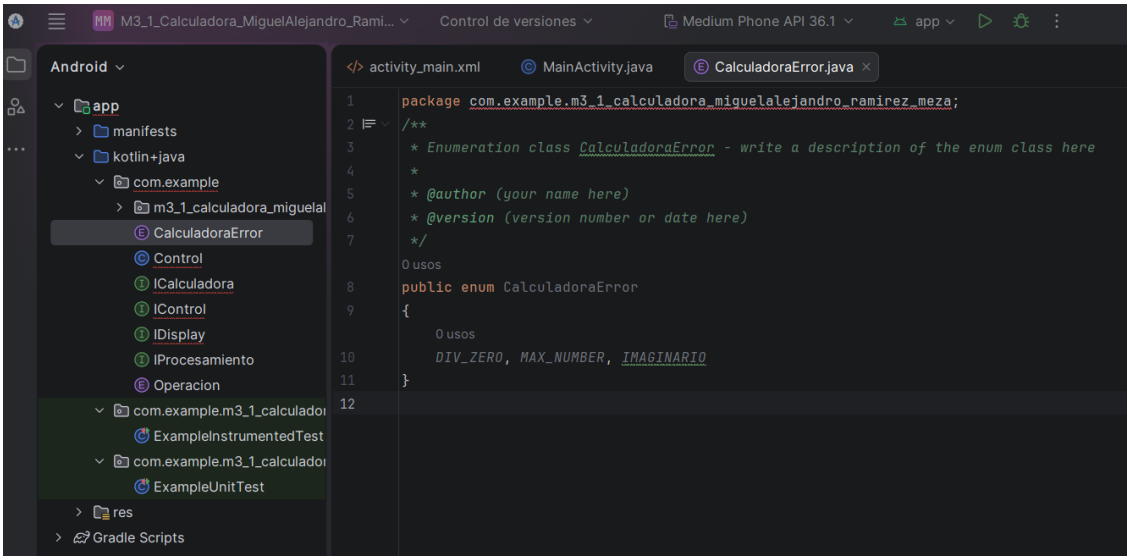
Al hacer esto se abre una carpeta en el explorador de Windows. En esta ruta debemos de copiar los archivos descargados de la calculadora



A continuación, debemos de verificar que al abrir cada archivo en Android Studio, la primera liga del código diga:
package com.example.m3_1_calculadora_miguelalejandro_ramirez_meza;



CalculadoraError



- Definir el código para cada archivo

Ahora ya podemos iniciar a desarrollar el código:

- CalculadoraError.java

  package com.example.m3_1_calculadora_miguelalejandro_ramirez_meza;

```
/**
 * Enumeration class CalculadoraError - write a description of the enum class here
 *
 * @author (Miguel Alejandro Ramirez Meza)
 * @version (1.0)
 */
public enum CalculadoraError
{
   DIV_ZERO, MAX_NUMBER, IMAGINARIO
}
```

- Control.java

```
package com.example.m3_1_calculadora_miguelalejandro_ramirez_meza;

/**
 * Write a description of class Control here.
 *
 * @author (Miguel Alejandro Ramirez Meza)
 * @version (1)
 */
public class Control implements IControl {
   private float x, y;
   private Operacion op;

   @Override public void introduceValorX(int x) { this.x = (float)x; }
   @Override public void introduceValorY(int y) { this.y = (float)y; }
   @Override public void introduceOperacion(Operacion op) { this.op = op; }
   @Override public float igual() {
      if (op == Operacion.SUM) return x + y;
      if (op == Operacion.RES) return x - y;
      if (op == Operacion.MUL) return x * y;
      if (op == Operacion.DIV) return (y == 0) ? Float.NaN : x / y;
      return 0;
   }
}
```

- Display.java

```
package com.example.m3_1_calculadora_miguelalejandro_ramirez_meza;

import android.util.Log;
```

```
/**
 * Implementación de la interfaz IDisplay para la Unidad 3.
 * @author Miguel Alejandro Ramirez Meza
 */
public class Display implements IDisplay {
    private static final String TAG = "CalculadoraUNAM";

    @Override
    public String muestraResultado(float res) {
        String mensaje = Float.isNaN(res) ? "Resultado: Error (NaN)" :
"Resultado: " + res;
        Log.d(TAG, mensaje);
        return mensaje;
    }

    @Override
    public String muestraError(CalculadoraError error) {
        String msg = "Error: " + error.name();
        Log.e(TAG, msg);
        return msg;
    }
}
```

- ICalculadora.java

```
package
com.example.m3_1_calculadora_miguelalejandro_ramirez_meza;
/**
 * Write a description of interface ICalculadora here.
 *
 * @author (Miguel Alejandro Ramirez Meza)
 * @version (1)
 */

public interface ICalculadora
{
    void setControl(IControl control);
    void setDisplay(IDisplay display);
    void setProcesamiento(IProcesamiento procesa);
}
```

- IControl.java

```
package
com.example.m3_1_calculadora_miguelalejandro_ramirez_meza;
/**
 * Write a description of interface IControl here.
```

```
 *
 * @author (Miguel Alejandro Ramirez Meza)
 * @version (1)
 */

public interface IControl
{
   void introduceValorX(int x);
   void introduceValorY(int y);
   void introduceOperacion(Operacion op);
   float igual();
}
```

- IDisplay.java

```
package
com.example.m3_1_calculadora_miguelalejandro_ramirez_meza;
/**
 * Write a description of interface IDisplay here.
 *
 * @author (Miguel Alejandro Ramirez Meza)
 * @version (1)
 */

public interface IDisplay
{
  String muestraResultado(float res);
  String muestraError(CalculadoraError error);
}
```

- IProcesamiento.java

```
package
com.example.m3_1_calculadora_miguelalejandro_ramirez_meza;
/**
 * Write a description of interface IProcesamiento here.
 *
 * @author (Miguel Alejandro Ramirez Meza)
 * @version (1)
 */

public interface IProcesamiento
{
   float suma(float x, float y);
   float resta(float x, float y);
   float multi(float x, float y);
   float div(float x, float y);
```

}

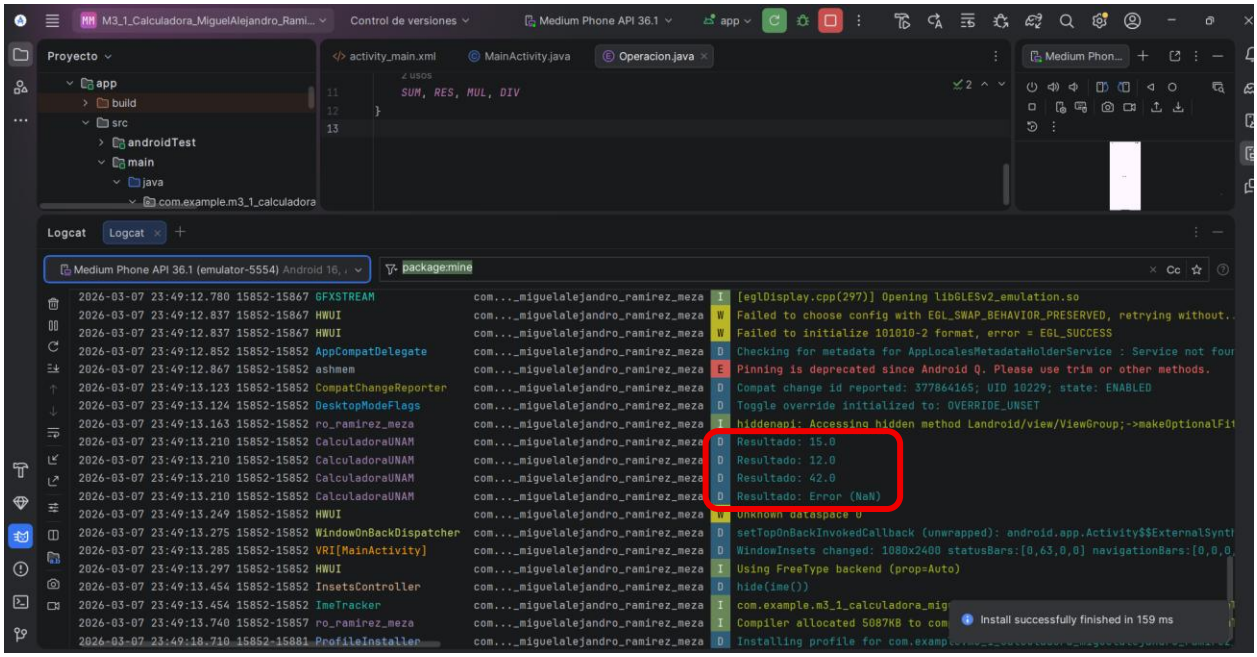
- MainActivity.java

```
package
com.example.m3_1_calculadora_miguelalejandro_ramirez_meza;

import android.os.Bundle;

import androidx.activity.EdgeToEdge;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.graphics.Insets;
import androidx.core.view.ViewCompat;
import androidx.core.view.WindowInsetsCompat;

public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_main);

ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main),
(v, insets) -> {
            Insets systemBars =
insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right,
systemBars.bottom);
            return insets;
        });
        // 1. Instanciamos los componentes (Implementación de interfaces)
        Control miControl = new Control();
        Display miDisplay = new Display();

        // 2. Realizamos las 4 operaciones básicas para probar la lógica

        // PRUEBA DE SUMA: 10 + 5
        miControl.introduceValorX(10);
        miControl.introduceValorY(5);
        miControl.introduceOperacion(Operacion.SUM);
        miDisplay.muestraResultado(miControl.igual());

        // PRUEBA DE RESTA: 20 - 8
        miControl.introduceValorX(20);
```

```
        miControl.introduceValorY(8);
        miControl.introduceOperacion(Operacion.RES);
        miDisplay.muestraResultado(miControl.igual());

        // PRUEBA DE MULTIPLICACIÓN: 6 * 7
        miControl.introduceValorX(6);
        miControl.introduceValorY(7);
        miControl.introduceOperacion(Operacion.MUL);
        miDisplay.muestraResultado(miControl.igual());

        // PRUEBA DE DIVISIÓN (Validación de NaN): 10 / 0
        miControl.introduceValorX(10);
        miControl.introduceValorY(0);
        miControl.introduceOperacion(Operacion.DIV);

        float resultadoDiv = miControl.igual();
        miDisplay.muestraResultado(resultadoDiv);
    }
}
```

- Operación.java

```
package
com.example.m3_1_calculadora_miguelalejandro_ramirez_meza;

/**
 * Enumeration class Operacion - write a description of the enum class
here
 *
 * @author (Miguel Alejandro Ramirez Meza)
 * @version (1)
 */
public enum Operacion
{
    SUM, RES, MUL, DIV
}
```



Conclusión

Esta actividad se me complicó, ya que primero tuve que investigar en que carpeta del proyecto tenía que adjuntar los archivos descargados. También me encontré con el problema de los errores ortográficos del uso de mayúsculas; y que, a pesar de tener el nombre idéntico en el package de cada archivo, Android Studio no hacía correctamente la relación lógica. Tuve que buscar una solución en Gemini, la cual era limpiar el proyecto (Clean Project) y posteriormente usar la opción de Reconstuir.

Por otro lado, me agrada la terminal Logcat y la detección de errores. Considero que es más amigable para el público general en comparación a VSCode ya que señala puntualmente los errores y las líneas donde se encuentran.